

CampusBot - College Information Chatbot

A chat-based assistant that answers student questions about admissions, courses, fees, timings, campus and exams, backed by a database and managed through an admin panel.

Author	Kikkara Jonus
Project Type	College-level Capstone Project
Domain	Chatbot / Natural Language Processing / Full-Stack Web Development
Status	Week 9 - scope, features, database structure, folder structure and development plan being finalised

Table of Contents

1. Project Overview
2. Objectives
3. Scope
4. Key Features
5. Technology Stack
6. System Architecture
7. Chatbot Query Flow
8. Database Design
9. NLP / Retrieval Pipeline
10. API Design
11. Repository Structure
12. Week 9-12 Roadmap
13. Testing Strategy
14. Deployment Plan
15. Getting Started
16. Risks and Mitigations
17. Future Enhancements
18. Author

1. Project Overview

Students and parents often struggle to find quick, accurate answers about a college. Details on admissions, eligibility, fees, timings, courses and exams are spread across notice boards, brochures, PDFs and websites, and the office cannot answer every repeated question.

CampusBot solves this by providing one chat window where:

- A student types a question in plain English, for example *"What is the fee for B.Tech CSE?"*
- The chatbot identifies what the student is asking (the **intent**) and which course or department it concerns.
- It looks up the answer in the college database and replies instantly.
- If it is not confident, it says so and points to the college office instead of guessing.
- Every question is logged, so administrators can see what students ask most and fill the gaps.

Design principle: start with a simple, reliable, fully explainable chatbot (rule and keyword matching over a database), then layer NLP improvements on top. The project must never depend on an external AI service to work.

2. Objectives

- Build a working college information chatbot with a clean chat interface.
- Store all college information in a database so that it can be updated without touching code.
- Answer questions on admissions, fees, courses, timings, campus, exams and faculty accurately.
- Provide an admin panel so staff can update information, manage FAQs and review unanswered questions.
- Record query history and show simple analytics to improve the chatbot over time.
- Follow sound engineering practice: modular code, tests, documentation and version control.

3. Scope

3.1 Capstone Version (In Scope)

Area	Included
Chat	Web chat interface, intent detection, keyword matching, FAQ responses, fallback replies
College data	College info, departments, courses, eligibility, fees, timings, location, exam schedule, faculty
Search	Search box for FAQs and information entries
Admin	Admin login and add / edit / delete for all information tables and FAQs
History	Query log with matched intent, confidence and answered / unanswered flag
Analytics	Top questions, top intents, unanswered questions, daily query counts
Retrieval	Rule + keyword matching first; TF-IDF with cosine similarity as the fallback layer

3.2 Out of Scope for This Version

- Dependence on an external LLM API for core answers (optional extension only)
- Voice input, mobile app and multi-language support
- Student accounts, login and personalised chat history
- Automatic website scraping
- Payment, admission form submission or any transactional features

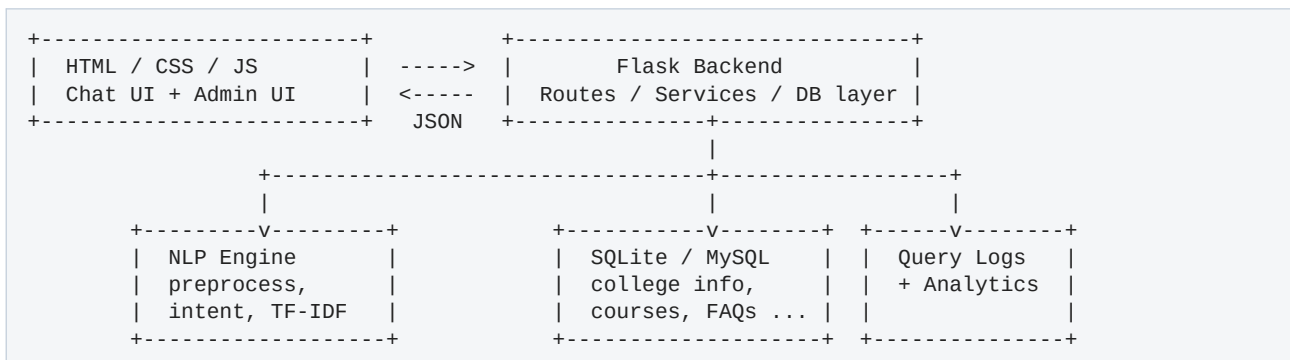
4. Key Features

Feature	Description
Chat-based interface	Message bubbles, typing indicator, quick-reply suggestion buttons
College information	About the college, vision, recognitions, contact details
Course / department info	Programmes offered, duration, seats, department overview
Admission and eligibility	Eligibility criteria, entrance exams, required documents, important dates
Fee information	Tuition, hostel, transport and other fees per course
College timings	Working hours, class timings, office and library hours
Campus and location	Address, directions, hostel, library, labs, canteen, transport
Academic / exam info	Academic calendar, exam timetable, result and revaluation notices
Faculty details	Faculty name, designation, department and contact
FAQ responses	Curated question and answer pairs matched by keywords or similarity
Search	Keyword search across FAQs and information entries
Database	All information stored in SQLite (or MySQL) and editable by admins
Admin panel	Secure panel to update data and manage FAQs
Query history and analytics	Every question logged; dashboard shows trends and unanswered queries
Safe fallback	When confidence is low, the bot says it does not know and suggests contacting the office

5. Technology Stack

Layer	Technology	Why this choice
Frontend	HTML + CSS + JavaScript	No build tooling needed; simple, portable and easy to demo
Backend	Python Flask	Lightweight, easy to learn, ideal for a REST-style chat API
Database	SQLite (MySQL optional)	Zero-setup file database for development; MySQL if multi-user hosting is needed
DB access	SQLite3 / SQLAlchemy	Parameterised queries prevent SQL injection
NLP	NLTK / basic Python text processing	Tokenising, stop-word removal and simple stemming
Matching	Rule + keyword matching	Fast, predictable and easy to debug
Advanced retrieval	TF-IDF + cosine similarity (scikit-learn)	Handles rephrased questions without any external API
Optional upgrade	LLM API	Can rewrite answers more naturally; kept optional so the project always works offline
Auth	Flask sessions + hashed passwords	Simple, secure enough for an admin-only panel
Version control	Git + GitHub	History, backup and collaboration

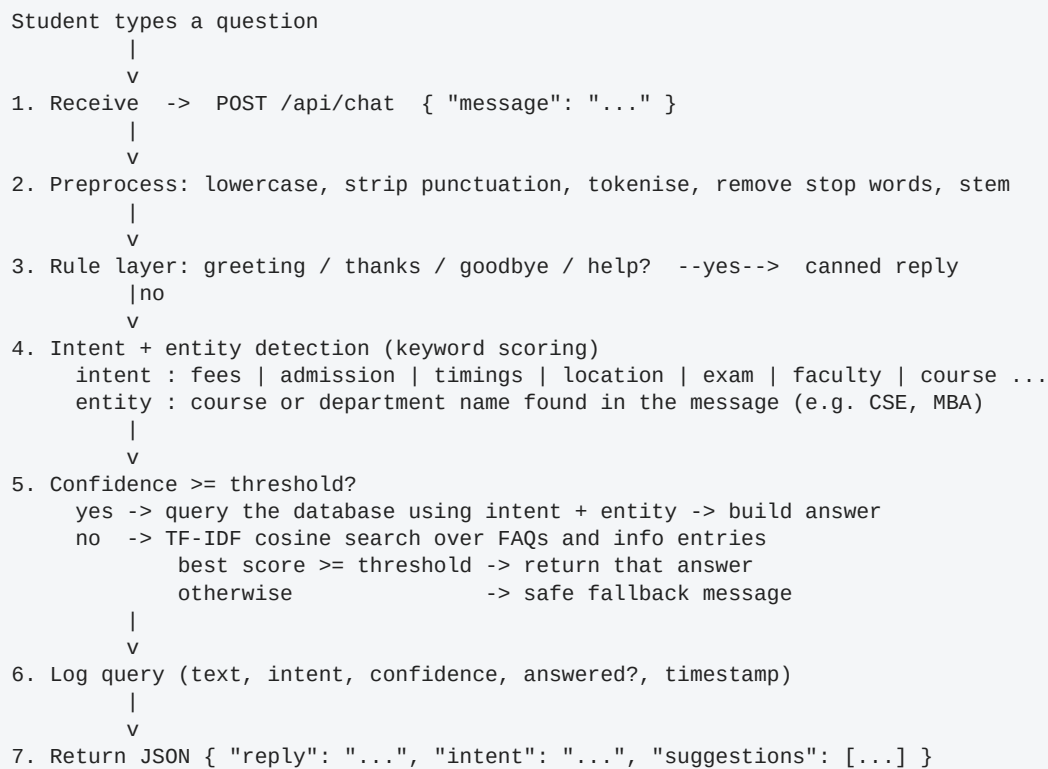
6. System Architecture



Backend layering: Route → Service → Database layer

- **Routes** handle HTTP requests, input validation and JSON responses.
- **Services** hold the business logic: chat handling, intent detection, answer building and analytics.
- **Database layer** contains all SQL access, so no query is written anywhere else.
- **NLP engine** is a standalone module with no web dependencies, which makes it easy to unit test.

7. Chatbot Query Flow



Example

Student asks	Detected	Bot response
What is the fee for B.Tech CSE?	intent = fees, entity = CSE	Annual tuition fee for B.Tech CSE is ... (from fees table)
When do the library hours start?	intent = timings, entity = library	The library is open from ... to ... (from timings table)
how to get admission	intent = admission	Admission process, eligibility and dates (from admissions table)
Who is the best cricketer?	no match, low similarity	Sorry, I don't have that information. Please contact the college office.

8. Database Design

Table	Purpose	Key columns
college_info	General college profile	id, name, about, address, city, state, phone, email, website, map_link
departments	Departments	id, name, description, hod_name, contact_email
courses	Programmes offered	id, department_id, name, level, duration_years, seats, description
eligibility	Admission criteria	id, course_id, criteria, entrance_exam, documents_required, important_dates
fees	Fee structure	id, course_id, fee_type, amount, academic_year, notes
timings	College / facility timings	id, facility, days, open_time, close_time, notes
facilities	Campus facilities	id, name, category, description, location
exams	Academic and exam schedule	id, course_id, semester, event, start_date, end_date, notes
faculty	Faculty details	id, department_id, name, designation, qualification, email
faqs	Curated Q and A pairs	id, question, answer, category, keywords
intents	Intent keyword lists	id, intent_name, keywords, response_template
query_logs	Chat history	id, user_query, matched_intent, confidence, answered, response, created_at
admins	Admin accounts	id, username, password_hash, created_at

Relationships

- **departments** 1 - N **courses** and 1 - N **faculty**
- **courses** 1 - N **fees**, **eligibility** and **exams**
- **query_logs** is standalone and append-only, so history is never lost

Notes

- Primary keys on every table; foreign keys with ON DELETE rules to avoid orphan rows.
- Indexes on courses.name, faqs.category and query_logs.created_at for fast lookups and reports.
- A schema.sql file and a seed.sql file create the tables and load sample data in one step.
- Passwords are stored only as salted hashes, never as plain text.

9. NLP / Retrieval Pipeline

9.1 Stage 1 - Preprocessing

- Lowercase the text and remove punctuation and extra spaces.
- Tokenise into words and remove stop words (is, the, of, what ...).
- Apply light stemming or lemmatisation so that *fees* and *fee* match.
- Normalise synonyms and abbreviations (CSE = computer science, hostel = accommodation).

9.2 Stage 2 - Rule and Keyword Intent Matching

- Each intent has a keyword list stored in the database (for example fees: fee, cost, tuition, charges, price).
- The score of an intent is the weighted count of its keywords found in the question.
- The highest scoring intent wins if its score passes the confidence threshold.
- Entity detection scans for known course and department names to refine the lookup.

9.3 Stage 3 - TF-IDF and Cosine Similarity (Fallback)

- All FAQ questions and information entries are converted into TF-IDF vectors at startup.
- The user's question is vectorised and compared with each entry using cosine similarity.
- The best match is returned only when its similarity passes a tuned threshold.
- Otherwise the safe fallback reply is returned and the question is flagged as unanswered.

9.4 Stage 4 - Optional LLM Upgrade

An LLM API can optionally rephrase the retrieved database answer into friendlier language. It only receives the retrieved facts, never free-form questions, and the system works fully without it.

10. API Design

Base path: /api

Method	Endpoint	Access	Description
GET	/health	Public	Service health check
POST	/chat	Public	Send a message, receive the bot reply
GET	/search?q=	Public	Search FAQs and information entries
GET	/faqs	Public	List FAQs, optionally by category
GET	/courses	Public	List courses and departments
POST	/admin/login	Public	Admin login, starts a session
POST	/admin/logout	Admin	End the admin session
GET/POST	/admin/{table}	Admin	List or create records (courses, fees, faqs ...)
PUT/DELETE	/admin/{table}/{id}	Admin	Update or delete a record
GET	/admin/queries	Admin	Query history with filters
GET	/admin/analytics	Admin	Top intents, top questions, unanswered count

Chat request / response

```
POST /api/chat
{ "message": "What is the fee for B.Tech CSE?" }

200 OK
{
  "reply": "The annual tuition fee for B.Tech CSE is ...",
  "intent": "fees",
  "confidence": 0.86,
  "suggestions": ["Hostel fee", "Eligibility", "Admission dates"]
}
```

11. Repository Structure

```
campusbot/
|-- app/
|   |-- __init__.py           # Flask app factory
|   |-- routes/              # chat.py, admin.py, public.py
|   |-- services/            # chat_service.py, analytics_service.py
|   |-- nlp/                 # preprocess.py, intent.py, tfidf.py
|   |-- db/                  # connection.py, queries.py
|   |-- templates/           # index.html, admin/*.html
|   |-- static/
|       |-- css/             # styles.css
|       |-- js/              # chat.js, admin.js
|-- database/
|   |-- schema.sql           # table definitions
|   |-- seed.sql             # sample college data and FAQs
|-- tests/                   # unit and API tests
|-- docs/                    # design notes, ER diagram, screenshots
|-- config.py
|-- requirements.txt
|-- run.py
|-- .env.example
-- README.md
```

12. Week 9-12 Roadmap

Week 9 immediate target: finalise the chatbot's scope, features, database structure, folder structure and development plan.

Week	Focus	Key deliverables	Status
9	Planning and setup	Scope and feature list frozen; database schema and ER diagram; folder structure; Flask skeleton with /health route; Git repository; this implementation plan	In progress
10	Core chatbot	SQLite schema and seed data; preprocessing module; keyword and intent matching; /chat endpoint; basic chat UI in HTML, CSS and JavaScript; fallback reply	Planned
11	Admin panel and history	Admin login; CRUD for courses, fees, timings, faculty, exams and FAQs; query logging; analytics page; TF-IDF fallback layer; search endpoint	Planned
12	Testing and delivery	Unit and API tests; accuracy check on a reference question set; bug fixes and UI polish; deployment; final report, README and demo	Planned

Week 9 task checklist

- Confirm the feature list with the project guide.
- Decide the database (SQLite for development) and finalise table definitions.
- Draw the ER diagram and chatbot flow diagram.
- Create the repository with the folder structure above.
- Prepare 30 to 50 sample questions and answers for seed data and later testing.
- Write the project plan (this document).

13. Testing Strategy

- **Unit tests** - preprocessing, intent scoring, TF-IDF matching and answer builders.
- **API tests** - each endpoint with valid, empty and malformed input using Flask's test client.
- **Database tests** - schema creation, seed loading and CRUD operations on a temporary database.
- **Chatbot accuracy check** - a reference set of 40+ questions covering every intent, with the expected intent and answer; target at least 85% correct.
- **Fallback tests** - irrelevant, empty and very long questions must return the safe fallback and not crash.
- **Admin security tests** - protected routes reject unauthenticated requests; SQL-injection strings are handled safely.
- **Usability check** - a few classmates try the chatbot and the unanswered questions are reviewed.

14. Deployment Plan

- Run locally for the capstone demo with the Flask development server.
- For a live demo, deploy to a free host such as PythonAnywhere or Render, using Gunicorn as the server.
- Keep secrets (secret key, admin credentials) in environment variables; never commit them.
- Back up the SQLite database file regularly; switch to MySQL if concurrent usage grows.
- Enable basic logging for errors and a /health endpoint for monitoring.

15. Getting Started

Prerequisites: Python 3.10+, Git, a modern web browser

```
# 1. Clone the repository
git clone https://github.com/<your-username>/campusbot.git
cd campusbot

# 2. Create a virtual environment and install dependencies
python -m venv venv
source venv/bin/activate           # Windows: venv\Scripts\activate
pip install -r requirements.txt

# 3. Configure environment
cp .env.example .env              # set SECRET_KEY and admin credentials

# 4. Create and seed the database
python -m app.db.init_db          # runs schema.sql and seed.sql

# 5. Start the application
python run.py
```

- Chat interface: <http://localhost:5000>
- Admin panel: <http://localhost:5000/admin>

16. Risks and Mitigations

Risk	Mitigation
Chatbot gives wrong or irrelevant answers	Answers come only from database records; confidence threshold plus safe fallback message
Students phrase questions in unexpected ways	Synonym lists, TF-IDF fallback and regular review of unanswered queries
College data becomes outdated	Admin panel for quick updates; last-updated dates shown on records
Incomplete or inconsistent source data	Validate data on entry; confirm sample data with the college office
Admin panel misuse or attack	Hashed passwords, session auth, parameterised SQL, input validation
Time pressure in a 4-week window	Fixed scope, weekly milestones, and a working basic chatbot first
Dependence on an external AI service	Core system is fully offline; LLM is optional only

17. Future Enhancements

- LLM-based natural rephrasing of answers (retrieval stays database-driven)
- Voice input and text-to-speech
- Multi-language support (for example Telugu and Hindi)
- Student login with saved chat history
- Notification of new circulars and exam notices
- Integration with the official college website or WhatsApp
- Machine-learning intent classifier trained on logged queries
- Feedback buttons (helpful / not helpful) to improve answers

18. Author

Kikkara Jonus

College Capstone Project - CampusBot: College Information Chatbot